

Revenant.AI — Engineering Status

Architecture, guarantees, known gaps, and the reasoning behind the non-obvious decisions.
Prepared for engineering review · 27 August 2026 · `main` @ `ac649cc`

21k

LINES PYTHON

176

TESTS (~1.3S)

8

MCP TOOLS

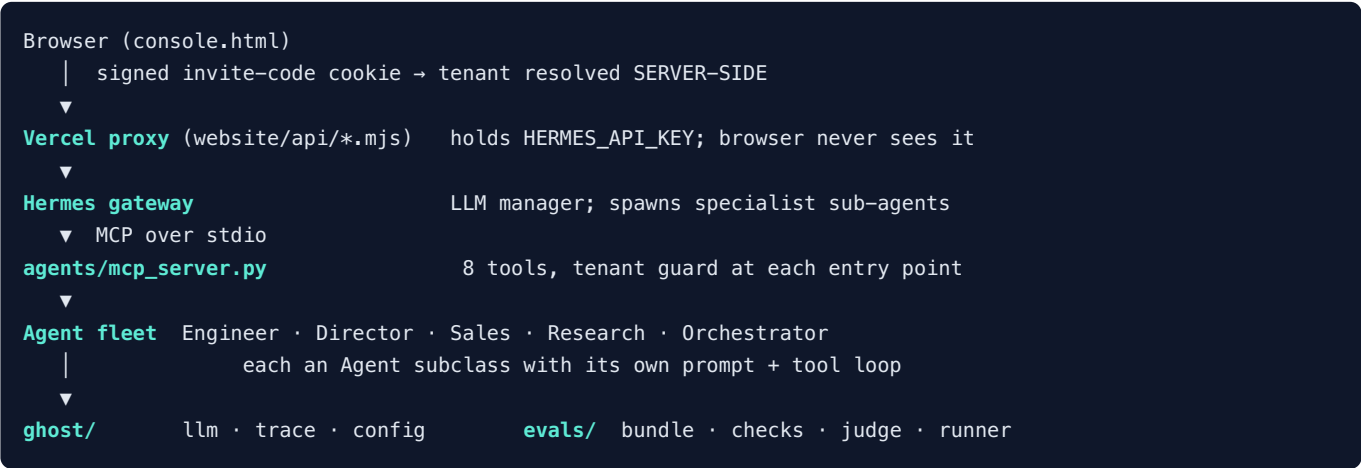
35

T1 CHECKS

125

COMMITTS

Architecture



Two layers of sub-agent, deliberately. The Hermes manager never calls a build tool itself — it spawns a `delegate_task` sub-agent per role ("You are the Engineer..."). That sub-agent calls one MCP tool, which instantiates the real Python `Engineer / Director / Sales` class — each running its own multi-step LLM tool-calling loop. The outer layer is orchestration; the inner is the actual work.

The guarantee that shapes everything

Never trust a tool's self-report. The dominant bug class in this codebase is "the tool returned success and the artifact is dead". It recurred often enough that the entire eval framework is built around structurally preventing it rather than catching it by review.

Concretely: every check re-fetches or re-opens the *real* artifact. Deterministic checks are hard gates — the LLM judge is never invoked on an artifact that already failed them, because a judge cannot rescue a dead link.

Verification tiers

Tier	What	LLM?	Guarantee
T0	Unit / contract tests	No	176 tests, offline, ~1.3s. Runs in CI.
T1	35 deterministic checks	No	Live URL, real audio+video streams, non-silent audio, working demo click, deck structure, email grounding. Hard gate.
T2	LLM judge	Yes	Rubric-scored, gated behind T1, every score requires a verified quote.
T3	Live end-to-end	Yes	Real campaigns against real prospects.

Why the judge can't fool itself

An LLM judge that hallucinates evidence is worse than no judge — it produces confident, unfounded numbers. Every criterion score must cite exact substrings from the real artifact. Citations are verified by literal substring match (whitespace/ case normalised, deliberately *not* fuzzy). **An unverified citation forces that criterion to zero**, regardless of the score claimed. The same mechanism is reused for learned preferences.

Multi-tenancy (4 phases)

Phase	Delivers	Design note
1	State isolation agents/tenancy.py	Tenant key = slug(startup) , derived from an argument the tools already carry — no new plumbing. Path traversal contained by re-slugging; 7 hostile inputs tested.
2	Per-tenant briefing cache	Tenant derived from ingested product_name , so all 14 summary() call sites benefit without change. Invalidated by a sha256 over ingested content.
3	Learned preferences	Quote-verified extraction (see above). Capped at 40 and deduped so prompts can't grow unbounded.
4	Authorization	REVENANT_PINNED_TENANT . Refuses cross-tenant calls rather than silently redirecting.

Read this before assuming the system is multi-tenant-safe. The tenant reaches the tools as an argument that travels *through an LLM*. An LLM can be talked out of an instruction. The console-side tenant injection is defence in depth, **not** a security boundary.

The real boundary is REVENANT_PINNED_TENANT : an environment variable set at process spawn, which cannot be argued with. The intended deployment is **one pinned MCP server per customer**. Verified over real MCP stdio: a pinned server refuses a call whose argument explicitly names another tenant.

Currently unpinned in the running deployment — safe today (single operator, single authenticated gateway), must be enabled before customer #2.

Observability

Pluggable tracing behind one shim (ghost/trace.py): jsonl by default, langfuse when configured — and jsonl keeps writing alongside it, so a misbehaving backend never loses tracing.

Verified live against a real Langfuse project. Traces carry: user_id = tenant, session_id = tenant:merchant (one campaign), tags, environment, git-sha release, specific observation types (agent / generation / retriever / evaluator), model and token usage for cost attribution.

Sensitive data is masked before leaving the process — emails, API-key shapes, bearer credentials. Confirmed on real data: a prospect address arrived as <email-redacted> . Masking is regex-over-payload rather than field-aware, because a field-aware masker only redacts the keys someone remembered to list.

Bugs worth knowing about

All found by *running* the system, not by reading it. Listed because each represents a class, not a one-off.

Bug	Why it mattered
Judge ran in offline mode	Returned a canned stub; the zero-score path skips citation checks by design, so a fake 0.0/100 was indistinguishable from a real harsh verdict. Real score was 22.5.
Score with no anchor	Every eval score silently 400'd into Langfuse — SDK logged it, the never-raise contract swallowed it. Found on the first live run.

Masking regex leak	<code>Authorization: Bearer <token></code> matched only through "Bearer"; the token survived in clear text. Caught by its own test.
Cross-thread span nesting	Every span inside a worker thread started a new root trace. 14 spans → 14 traces; now 1.
Dotted invite code	Accepted at login with a 200, then 401'd forever. Silent permanent lockout.
Gateway outlived its MCP subprocess	Live console reported "tool not available". Operational failure mode — worth alerting on.
Deck CTA check was a coin-flip	Two near-identical decks landed on opposite sides purely by synonym choice. A bad check is worse than none — it would have fed the improvement loop false signal.
Tests read real user state	Two tests passed against the developer's actual <code>~/ .revenant</code> instead of their fixtures.

Known gaps

Gap	Severity	Detail
Hosting = temporary tunnel	High	Free-tier single-tunnel limit; links expire. Detected correctly by T1, but not durable. Blocks launch.
Pinning not enabled	High	Supported and tested; not deployed. Required before multi-customer.
Self-heal retries	Medium	<code>extra_instruction</code> plumbing exists; retry orchestration not built.
CI is T0-only	Medium	No browser/network in CI by design. T1 runs locally via <code>make eval</code> .
Trace timestamps	Low	Batch-emitted at flush; real timings preserved in metadata, not on the UI timeline.
Lint debt	Low	~400 pre-existing findings, mostly <code>ISC004</code> . Ruff non-blocking in CI.

Operational notes

- **Secrets:** `.env` (gitignored, mode 600). Browser never receives a key — the Vercel proxy holds them. **The current Langfuse key pair was pasted into a chat transcript and should be rotated.**
- **Gateway health:** it can outlive its MCP subprocess and report tools as missing. A restart fixes it; this deserves monitoring.
- **Degraded mode is normal:** missing enrichment keys degrade gracefully (guessed emails, macOS voice, no payment link) rather than failing — check `status` before assuming a feature is broken.
- **Prompt changes:** bump `PROMPT_VERSION` and regenerate the baseline; a T0 guard enforces this.

Where to start reading

Path	Why
<code>docs/evals-observability-design.md</code>	The architecture rationale. Read first.
<code>agents/tenancy.py</code>	Multi-tenancy + the trust boundary, stated plainly.
<code>evals/judge.py</code>	Citation verification (M1–M6).
<code>agents/mcp_server.py</code>	All 8 tools; the API surface.
<code>ghost/trace.py</code>	Tracing shim + cross-thread propagation.

Code comments explain *why*, and frequently cite the specific bug that motivated the decision — they are the primary documentation for non-obvious choices.